

AEGIS — Architecture Plan

Anonymization & Exchange Gateway for Imaging Studies

Matthew L. Senjem, M.S. | February 18, 2026

Why "AEGIS"?

The name **AEGIS** serves double duty. As an acronym, it describes exactly what the system does: an **A**nonymization & **E**xchange **G**ateway for **I**maging **S**tudies. The word itself comes from Greek mythology — the aegis was the shield of Zeus and Athena, a symbol of protection. This captures the platform's core mission: shielding patient identity while enabling the free flow of medical imaging data for research and clinical care.

Context

Medical imaging studies across **all DICOM modalities** need to be shared between hospitals, universities, and research institutions. Before transmission, DICOM images must be de-identified of all PHI per HIPAA Safe Harbor rules (18 identifier categories). Head imaging additionally requires **defacing** (removing facial features from 3D volumes to prevent re-identification via facial reconstruction). Sending hospitals have locked-down IT environments where installing software is difficult or impossible.

AEGIS is a GCP-hosted platform with **two ingress paths** into the enterprise GCP tenancy:

1. **External-site ingress** — external institutions upload DICOM data of any modality after browser-based tag anonymization.
2. **Internal-enterprise ingress** — studies originating within the enterprise network are ingested directly into the same tenancy.

Both paths converge on a common processing pipeline (validation, de-identification policy enforcement, optional defacing, QC, and audit), after which approved data is routed/shared to authorized downstream recipients.

Supported Modalities

AEGIS supports **all DICOM-compliant imaging modalities**, including but not limited to:

- **MRI** — brain, spine, cardiac, musculoskeletal, abdominal, breast
- **CT** — head, chest, abdomen/pelvis, cardiac, angiography
- **PET** — FDG, amyloid, tau, PSMA, whole-body
- **PET/CT and PET/MR** — fused multimodal studies
- **Ultrasound** — echocardiography, abdominal, vascular
- **X-Ray / CR / DR** — chest, extremity, mammography (FFDM/DBT)
- **Nuclear Medicine** — SPECT, planar scintigraphy
- **Fluoroscopy** — GI, interventional
- **Secondary Capture / Structured Reports / RT** — dose reports, annotations, treatment plans

The de-identification engine operates at the DICOM tag level per PS3.15 Annex E and is **modality-agnostic** — it works identically for any DICOM data.

MVP Focus

- **Brain MRI** (T1, T2, FLAIR, DWI, fMRI, etc.)
- **Brain PET** (FDG, Amyloid, Tau tracers)

- **Brain CT**

Brain imaging is the MVP focus because it exercises the most complex pipeline (tag de-identification + automated defacing). All other modalities work through the same upload and de-identification flow but skip the defacing step.

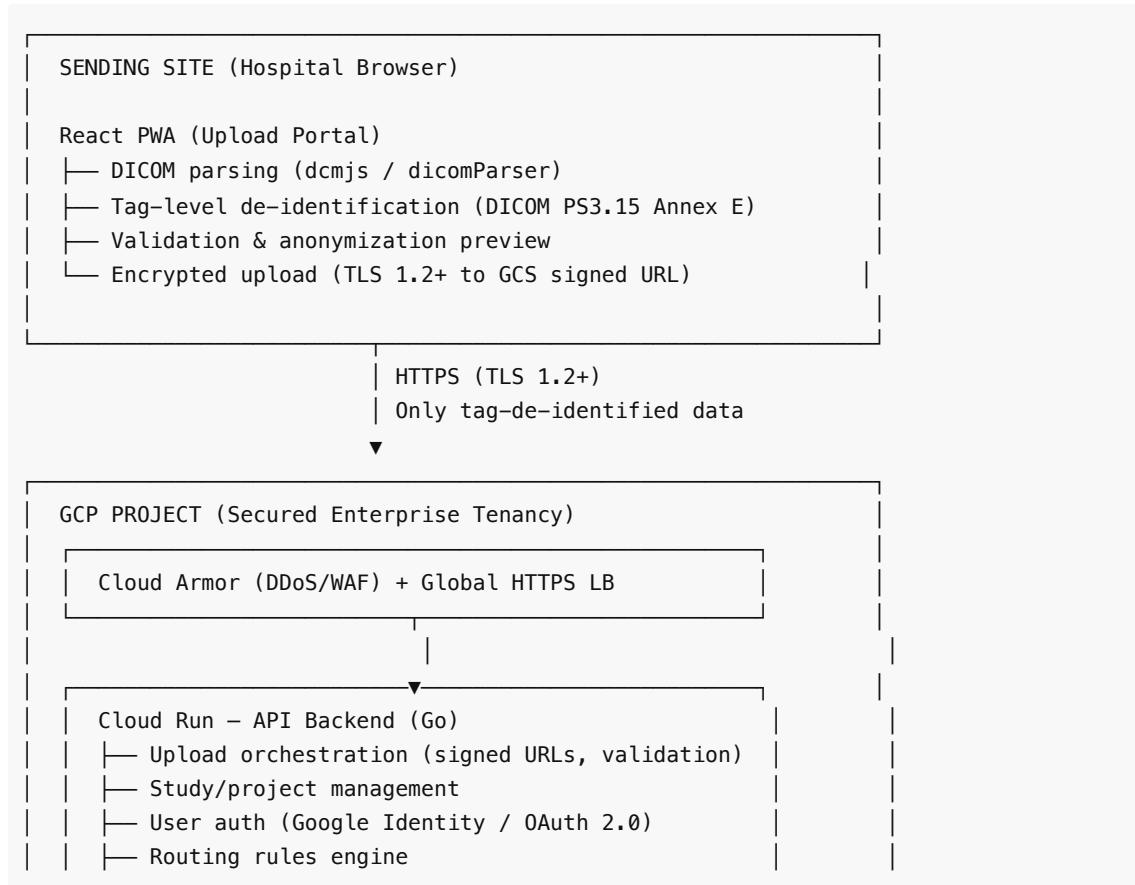
Future Scope

- Non-DICOM formats (pathology whole-slide imaging, electron microscopy)
- DIMSE (non-web) protocol support for legacy PACS integration

Design Principles

- **Open source** tools and libraries wherever possible
- **Minimal vulnerability surface** — prefer compiled languages (Go) over interpreted (Python) for backend services to reduce dependency sprawl and CVE exposure
- **Client-side tag anonymization** — PHI stripped in the browser before upload
- **Server-side defacing** — facial feature removal from head imaging after upload (too compute-intensive for browser); other modalities pass through without defacing
- **Zero-install at sending sites** — pure web app, no browser extensions, no desktop software required
- **Multi-cloud** — application layer is cloud-agnostic (GCP, AWS, or Azure); storage, auth, and container orchestration abstracted behind pluggable interfaces; Terraform modules provided for GCP and AWS

System Overview



└ DICOMweb proxy to Healthcare API

Cloud
Storage
(staging)

Healthcare API
└ DICOM Store
└ DICOMweb
└ De-id
└ Pub/Sub
└ BigQuery

Cloud Run – Defacing
Service (Python)
└ dcm2niix
└ mri_deface or
DeepDefacer
└ Pixel injection

Cloud Run – Admin Dashboard (React + nginx)

└ Behind Identity-Aware Proxy (IAP)
└ Study review / QC interface
└ Defacing review (before/after)
└ OHIF Viewer integration (DICOMweb)
└ Audit logs

Cloud SQL (PostgreSQL 15)

└ User accounts, RBAC, institutions
└ Upload sessions & routing rules
└ App-level audit trail

BigQuery

└ Healthcare API DICOM metadata export
└ Audit analytics & QC dashboards

Vertex AI

└ Burned-in PHI detection (Document AI / OCR)
└ Image QC & smart routing models

Email (dual-path)

└ Internal: PSC → On-Prem SMTP (admins)
└ External: SendGrid (uploaders)

VPC Service Controls perimeter around all services
CMEK encryption via Cloud KMS

Repository Structure (5 Repos)

1. `aegis-terraform-prj` — GCP Project Bootstrap

- Terraform for project-level resources: APIs, billing, org policies
- Service account creation, IAM bindings
- VPC Service Controls perimeter
- Cloud KMS key rings and keys
- BAA configuration documentation

2. `aegis-terraform-infra` — Infrastructure

- VPC, subnets, Cloud NAT, firewall rules
- Cloud Run service definitions (API, admin dashboard, defacing service)
- Cloud Storage buckets (staging, archive)
- Healthcare API dataset + DICOM stores (raw + defaced)
- Cloud SQL (PostgreSQL 15) instance for application data
- BigQuery dataset for DICOM metadata analytics and audit reporting
- Vertex AI API enablement and endpoint configuration
- Private Service Connect endpoint for on-prem SMTP relay
- Cloud Armor security policies
- Pub/Sub topics and subscriptions
- Cloud Build triggers
- IAP configuration for admin dashboard
- Artifact Registry for Docker images
- Monitoring, alerting, audit log sinks

3. `aegis-api` — Backend API (Go)

- Cloud Run service, `distroless` Docker image (~10-20 MB)
- Endpoints:
 - `POST /api/upload/init` — generate signed URL for GCS upload
 - `POST /api/upload/complete` — trigger ingest pipeline
 - `GET/POST /api/studies` — study management
 - `GET/POST /api/projects` — project/routing config
 - `GET /api/dicomweb/*` — proxy to Healthcare API DICOM store
 - `POST /api/deface/{studyUID}` — trigger defacing pipeline
 - `GET /api/audit` — audit log queries
 - `POST /api/notify` — trigger email notifications via SMTP/PSC
- Auth middleware (JWT / Google Identity tokens)
- Pub/Sub event handlers for DICOM ingest events
- Cloud SQL (PostgreSQL) for users, projects, routing rules, audit trail
- Vertex AI client for burned-in PHI detection and QC models
- SMTP client for email notifications via Private Service Connect

4. `aegis-frontend` — React Applications

- Monorepo with two apps + shared libraries
- **Upload Portal** (public-facing):
 - DICOM file selection (drag-and-drop, directory picker via `<input webkitdirectory>`)
 - Client-side parsing with `dcmjs/dicomParser` in Web Workers
 - De-identification engine (DICOM PS3.15 Annex E tag actions)
 - Anonymization preview (before/after tag diff table)

- Study metadata summary (modality, series count, image count)
- Progress tracking, chunked upload to GCS via signed URLs
- PWA capabilities for optional "install to desktop"
- **Admin Dashboard** (internal, behind IAP):
 - Study browser with OHIF Viewer integration
 - QC review workflow (prearchive → defacing → archive)
 - Defacing review: side-by-side before/after 3D rendering
 - Routing rule configuration UI
 - User/institution management
 - Audit log viewer

5. **aegis-client** — Uploader Library (TypeScript)

- Reusable TypeScript library extracted from the Upload Portal
 - Can be embedded in other web apps or used standalone
 - Core modules: DICOM parser, de-identification engine, upload client
 - Published as npm package for institutional integration
 - No desktop app — pure browser library
-

Technology Choices

Backend: Go

Rationale for Go over Python for the main API:

- **Smaller vulnerability surface:** Single static binary, no runtime, no pip dependency tree. `distroless` or `scratch` Docker image ~10-20 MB vs ~200-500 MB for Python
- **Fewer CVEs:** Go stdlib covers HTTP, JSON, crypto, TLS. No equivalent of the constant Python/Alpine package churn
- **Cloud Run fit:** ~100ms cold starts vs ~2-5s for Python, lower memory
- **Concurrency:** Goroutines for concurrent upload handling without GIL
- **DICOM library:** [suyashkumar/dicom](#) for validation/metadata. The **Healthcare API handles heavy DICOM processing**, so the Go library only needs basic parsing
- **GCP SDK:** First-class Go SDK (`cloud.google.com/go/healthcare` , `cloud.google.com/go/storage`)

Python sidecar (defacing service only):

- All defacing tools (`mri_deface`, `DeepDefacer`, `pydeface`) are Python or C with Python bindings
- Isolated Cloud Run service with its own container and update cycle
- Keeps the Python dependency surface separate from the main API
- Uses `pydicom` for pixel data injection (defaced NIfTI → original DICOM)

Frontend: React + TypeScript

- **Key libraries:**
 - `dcmjs` (MIT) — DICOM read/write in browser
 - `dicomParser` (MIT) — robust DICOM Part 10 parsing
 - OHIF Viewer components (MIT) for admin dashboard
- Web Workers for background DICOM processing (keeps UI responsive)

Infrastructure: Terraform (Multi-Cloud)

- **GCP:** terraform/project/ (bootstrap) + terraform/infra/ (Cloud Run, Healthcare API, Cloud SQL, GCS)
- **AWS:** terraform/aws/ (VPC, ECS Fargate, RDS, S3, ALB, ECR, KMS, SNS/SQS)
- **Azure:** Planned
- CI/CD: GitHub Actions (Go build+vet, Python syntax, TypeScript type check, Docker build)
- Docker images stored in Artifact Registry (GCP) or ECR (AWS)

DICOM Storage: GCP Healthcare API

- **DICOMweb** (STOW-RS, WADO-RS, QIDO-RS) — no need to self-host dcm4chee or Orthanc
- **Two DICOM stores:** raw (tag-de-identified but not defaced) and clean (fully processed)
- **Built-in de-identification** as server-side validation pass
- **Pub/Sub** notifications on ingest → trigger defacing pipeline
- **BigQuery export** for metadata analytics and QC reporting

Application Database: Cloud SQL (PostgreSQL 15)

Rationale: Healthcare API stores DICOM data and metadata, but the application needs a relational database for state that doesn't belong in DICOM:

- **User accounts and RBAC** — roles (uploader, reviewer, admin), institution membership
- **Projects and institutions** — multi-tenant organization of studies
- **Upload sessions** — tracking upload state, resumability, completion
- **Routing rules** — configurable forwarding rules per project/modality
- **App-level audit trail** — who did what, when (supplements GCP audit logs)
- **Email notification log** — delivery tracking, digests

Cloud SQL with Private IP inside the VPC. CMEK encryption via Cloud KMS.

Analytics: BigQuery

- **Healthcare API DICOM metadata export** — automatic streaming of study/series/instance metadata
- **Audit analytics** — query upload/anonymization/routing history for compliance
- **QC dashboards** — aggregate stats on processing times, rejection rates, data quality
- **Research metadata queries** — cross-study searches for specific scan parameters, modalities

AI/ML: Vertex AI

- **Burned-in PHI detection** — Document AI / Vision API to OCR pixel data and flag text embedded in images
- **Image quality assessment** — custom models for detecting motion artifacts, incomplete coverage, truncation
- **Smart routing** — classify studies by modality/anatomy when DICOM metadata is unreliable or missing
- **Future:** defacing quality scoring, automated QC pass/fail

Email Notifications

Current implementation (api/email/):

- Single SMTP client using Go stdlib net/smtp — zero external dependencies
- Disabled by default; enabled by setting SMTP_HOST env var (silent no-op when unset)
- Triggers implemented: share created → recipient; upload confirmed → uploader; study approved/rejected → uploader
- No PHI in email bodies — only anonymized Study UUIDs, file counts, and export URLs

- Dev: use [Mailpit](#) (`docker run -p 1025:1025 -p 8025:8025 axllent/mailpit`)

Planned dual-path (production):

Internal recipients (admins, reviewers) — On-prem SMTP via Private Service Connect:

- PSC endpoint from GCP VPC to on-prem SMTP relay
- Already compatible: `net/smtp` client routes through the PSC endpoint with no code changes
- Additional triggers: defacing alerts, routing failure alerts, weekly/monthly digests

External recipients (uploaders, sending sites) — SendGrid:

- External users cannot receive email through the on-prem relay
- **SendGrid** for external delivery (bounce handling, spam compliance, analytics)
- SendGrid exposes an SMTP interface — no code changes required, just point `SMTP_HOST` at the SendGrid relay

Viewing: OHIF Viewer

Local dev: OHIF v3 runs as a Docker container on `:3002` via `docker compose up ohif` . Configured via `ohif-config.js` (repo root) to use the Go API's DICOMweb proxy at `http://localhost:8080/dicomweb` .

DICOMweb proxy (`api/handler/dicomweb.go`): minimal QIDO-RS + WADO-RS implemented in Go without a DICOM library. Serves study/series/instance metadata from PostgreSQL and streams raw DICOM bytes from local storage. Uses fake deterministic UUIDs (`{studyUID}.1.{fileIndex}`) that map directly to file paths (`dicom/{store}/{studyUID}/{index}.dcm`).

Admin dashboard **View button**: each study row shows an inline iframe panel (OHIF embedded in the dashboard) and an "Open in new tab ↗" link. Both modes open `http://localhost:3002/viewer?StudyInstanceUIDs={uid}` .

Production: OHIF served from Cloud Run behind IAP, configured to connect directly to the GCP Healthcare API DICOMweb endpoint. The Go DICOMweb proxy is replaced by the Healthcare API's native DICOMweb support.

- Web-based, React, MIT license
- Supports all standard DICOM modalities (MRI, CT, PET, US, X-Ray, NM, etc.)

Anonymization Architecture

Two-Phase De-identification

Phase 1 — Client-side tag anonymization (browser, before upload):

- Tag-level de-id per DICOM PS3.15 Annex E Basic Profile
- Action codes per tag: D (dummy), Z (zero), X (remove), U (replace UID), C (clean)
- All 18 HIPAA Safe Harbor identifier categories addressed
- Key tags handled:
 - PatientName, PatientID, PatientBirthDate → zeroed/replaced
 - InstitutionName, ReferringPhysician, OperatorsName → removed
 - StudyInstanceUID, SeriesInstanceUID, SOPInstanceUID → deterministic hash (preserves longitudinal linkage)
 - StudyDate, SeriesDate → date-shifted with preserved temporal relationships

- All private tags → removed by default (retain list for known-safe manufacturer tags needed for research, e.g., diffusion gradient tables)
- Configurable profiles: "Research (full)" vs "Institutional (partial)"
- **Anonymization preview:** user sees before/after tag diff before confirming
- Mapping table (original → anonymized IDs) stays local, never uploaded

Phase 2 — Server-side processing (after upload):

- Healthcare API validates de-identification completeness
- InfoType detection (DLP-based) catches any residual PHI in text fields
- **Defacing pipeline** (see below) removes facial features from head scans
- Results written to `clean` DICOM store; `raw` store used as staging

Defacing Pipeline (Server-Side — Head Imaging Only)

Defacing applies **only to head/brain imaging** (MR, PT, CT with BodyPartExamined = HEAD or BRAIN). All other modalities and anatomies bypass defacing and proceed directly to the `clean` store after tag de-identification validation.

Pipeline:

1. Pub/Sub event: new study ingested into Healthcare API
2. Go API checks modality and body part
3. If head imaging (MR/PT/CT + HEAD/BRAIN) → triggers defacing service
If non-head imaging → moves directly to "clean" store after validation
4. Defacing service (head imaging only):
 - a. Retrieves DICOM series from Healthcare API via WADO-RS
 - b. Converts to NIfTI via dcm2niix
 - c. Runs defacing tool (`mri_deface` or `DeepDefacer`)
 - d. Injects defaced pixel data back into original DICOM (`pydicom`)
 - e. Stores defaced DICOM back via STOW-RS to "clean" store
5. Admin reviews defacing quality in dashboard (OHIF side-by-side)
6. Admin approves → study available for routing/download

Defacing tool options (ranked by deployment practicality):

Tool	Docker Size	Time/Volume	Quality	License	Best For
mri_deface (FreeSurfer)	~0.5 GB	2-10 min	Good	Free (no license needed)	MVP — smallest footprint
DeepDefacer	~1-2 GB	~2 min	Good	MIT	Speed-optimized alternative
pydeface	~3-6 GB	2-10 min	Good (83%)	MIT	If FSL already available
mri_reface	~2-4 GB	>1 min	Best identity protection	Non-commercial only	Research-only deployments

afni_refacer	~3-5 GB	~30 min	Best overall (89%)	Public domain	Quality-critical cases
MiDeFace (FreeSurfer)	~4-8 GB	~30 min	Very good	FreeSurfer license	When accuracy is paramount

Recommendation for MVP: Start with **mri_deface** (standalone, ~0.5 GB Docker image, no license required, 2-10 min/volume). Upgrade to **DeepDefacer** if faster processing is needed, or **afni_refacer** if quality is paramount.

Modality-specific defacing needs:

- **Brain MRI:** All defacing tools work well. Primary use case.
- **Brain PET:** mri_reface explicitly supports Amyloid PET, Tau PET, FDG PET. Others may need co-registered MRI for alignment.
- **Brain CT:** mri_reface supports CT. Others may require adaptation. Less common for research sharing.

Key dependencies for defacing service container:

- `dcm2niix` (static binary, ~2 MB) — DICOM to NIfTI conversion
- `mri_deface` + atlas files (~0.5 GB) — defacing
- `pydicom` + `nibabel` — pixel data injection back into DICOM
- `FastAPI` / `uvicorn` — lightweight HTTP server
- Base image: `python:3.12-slim` or Alpine-based for minimal CVE surface

Data Flow

1. Data enters through one of two ingress paths:
 - **External-site path:** uploader uses ``upload.aegis.example.com``, browser performs client-side tag anonymization, then uploads de-identified files.
 - **Internal-enterprise path:** internal system/user ingests studies directly into enterprise-controlled intake (API/DICOMweb/batch ingest).
2. Go API records intake session and metadata in PostgreSQL audit/application tables.
3. Ingested files are written to staging / intake and then into Healthcare API DICOM store.
4. Pub/Sub notification triggers common processing pipeline:
 - a. Server-side de-id validation (Healthcare API)
 - b. If head imaging → trigger defacing service
If non-head → move directly to "clean" store
 - c. (Head only) Defacing service processes and stores to "clean" store
5. Admin reviews in dashboard (OHIF Viewer)
 - Tag anonymization completeness check
 - Defacing quality review (head imaging only – before/after)
6. Admin approves → study becomes shareable/exportable under policy.
7. External partner access is granted via controlled mechanisms (signed download links, authorized DICOMweb routes, or approved project-level export channels).

Security Architecture

Layer	Measure
Network	VPC Service Controls perimeter, Cloud Armor DDoS/WAF, Global HTTPS LB
Transport	TLS 1.2+ enforced on all endpoints (automatic on GCP)
Auth (upload portal)	OAuth 2.0 / Google Identity — sending sites get project-scoped credentials
Auth (admin dashboard)	Identity-Aware Proxy (IAP) — Google Workspace / Cloud Identity
Authorization	Per-service-account IAM (least privilege); app-level RBAC for projects
Encryption at rest	CMEK via Cloud KMS for Healthcare datasets, GCS buckets
Audit	Cloud Audit Logs for all data access; app-level audit trail
PHI protection	Client-side tag de-id; server-side validation + defacing; no PHI in logs
Container security	Go: distroless base (~10 MB). Python defacer: slim base, pinned deps
Database	Cloud SQL PostgreSQL 15 with Private IP, CMEK encryption, automated backups
Email	SMTP via Private Service Connect to on-prem relay (no PHI in email bodies)
AI/ML	Vertex AI within VPC-SC perimeter; no data leaves project boundary
Secrets	Secret Manager for API keys, service credentials, DB passwords
Scanning	Artifact Registry vulnerability scanning; regular dependency updates

Comparison to Existing Platforms

Aspect	AEGIS	XNAT	Flywheel	LONI IDA	MIRC CTP
Hosting	Multi-cloud (GCP, AWS, Azure)	Self-hosted	Commercial SaaS	On-prem	On-prem gateway
Client de-id	Browser (zero install)	Electron desktop app	CLI / Edge connector	Java desktop app	On-site Java app
Server de-id	Healthcare API + defacing svc	DicomEdit scripts	Built-in	Post-upload	Pipeline stages
Defacing	Automated server-side	Manual or plugin	Built-in	Separate	Not included
DICOM store	Healthcare API (DICOMweb)	PostgreSQL + filesystem	MongoDB + S3	MySQL + Isilon	Filesystem

Viewer	OHIF (embedded)	Built-in viewer	Built-in viewer	Web viewer	None
Open source	Yes	Yes	No	Partial	Yes
Install at site	None	Desktop client (optional)	CLI (optional)	Java app (required)	Java app (required)
Modalities	All DICOM	Neuroimaging	All imaging	Neuroimaging	All imaging

Key differentiators:

1. Zero-install browser-based anonymization — no Java, no Electron, no CLI
2. Modality-agnostic: works with any DICOM data, not limited to a single specialty
3. Multi-cloud — runs on GCP, AWS, or Azure with Terraform modules for each; no cloud lock-in
4. Automated server-side defacing pipeline for head imaging
5. Pluggable AI backends — local (Tesseract, pydicom) for dev, cloud AI (Vertex AI, SageMaker) for production
6. Minimal vulnerability surface (Go backend, distroless containers)

Phased Implementation

Phase 1: Foundation (MVP)

1. **Terraform (prj)**: GCP project, APIs, IAM, KMS, VPC-SC
2. **Terraform (infra)**: VPC, Cloud Run, GCS, Healthcare API, Cloud SQL (PostgreSQL), BigQuery dataset, Cloud Armor, Pub/Sub
3. **Cloud SQL schema**: Users, projects, institutions, upload sessions, audit trail
4. **Go API**: Signed URL generation, upload orchestration, STOW-RS ingest, basic auth, PostgreSQL integration
5. **Upload Portal**: DICOM file picker, tag-level de-id engine, preview, chunked upload
6. **Admin Dashboard**: Study list, OHIF viewer, basic QC accept/reject
7. **BigQuery**: Healthcare API metadata export, basic audit queries

Phase 2: Defacing Pipeline

8. **Defacing service**: Python Cloud Run, mri_deface + dcm2niix, DICOM pixel injection
9. **Admin defacing review**: Before/after comparison in dashboard
10. **Automated trigger**: Pub/Sub → detect head imaging → trigger defacing
11. **Email notifications**: PSC to on-prem SMTP, upload confirmations, defacing alerts

Phase 3: Operations & Routing

12. **Routing rules engine**: Destinations table + RoutingRules table; priority-ordered evaluation on every ingest; actions: `require_defacing`, `auto_approve`, `require_qa`, `reject`, `route_to` (async DICOMweb forward); routing log per study; admin UI in Routing tab (PR #12)
13. **Institution management**: Institutions table + institution_projects join table; institution_id on studies/sessions; CRUD API + admin UI in Institutions tab; project-scoped roles (sender/receiver/admin) (PR #13)
14. **Per-project anonymization profiles**: Multiple named profiles per project; `retained_tags` JSONB list overrides Basic Profile strip/zero actions client-side; `default_anon_profile_id` on projects; upload portal fetches active profile at upload time; admin Profiles tab (PR #15)

15. **Audit log viewer:** Full trail from PostgreSQL + filterable UI in admin dashboard (PR #7)
16. **Email digests:** `digest_subscriptions` table; per-project weekly/monthly subscriptions; hourly scheduler goroutine sends plain-text summaries (study counts, share activity, no PHI); admin Notifications tab (PR #17)
17. **Admin user management:** `admin_users` table (migration 011); CRUD API; admin Users tab with role (admin|viewer), enable/disable, audit logging (PR #20)
18. **Project settings UI:** GET/PUT `/api/projects/{id}` ; slug auto-generation on create; admin Projects tab with create/edit; `project.created` / `project.updated` audit entries (PR #21)
19. **Upload portal QoL:** `onFileStart` callback in `@aegis/client` ; per-file filename display below progress bar; per-file PUT auto-retry (3x with exponential backoff) (PR #22)

Phase 4: Advanced Processing (Vertex AI)

20. **Burned-in PHI detection:** Python OCR service (`phi-detection/`) with Tesseract backend (local dev) and Vertex AI Document AI (production); `require_phi_scan` routing rule action; `phi_scan_required` / `phi_scan_status` study fields; async dispatch from Go API; admin dashboard PHI scan badge + scan button (PR #29)
21. **QC automation:** Python QC service (`qc-service/`) with pydicom+numpy backend (local dev) and Vertex AI (production); `require_qc_check` routing rule action; `qc_required` / `qc_status` study fields; 5 automated checks (file integrity, slice consistency, SNR, coverage, missing slices); async dispatch from Go API; admin dashboard QC badge + Run QC button (PR #32)
22. **Smart routing:** Python classification service (`classification-service/`) with heuristic DICOM tag analysis backend (local dev) and Vertex AI (production); `require_classification` routing rule action; `classification_required` / `classification_status` study fields; classifies modality + body_part from DICOM headers (SOP Class UID, SeriesDescription, BodyPartExamined patterns); updates study metadata and **re-evaluates routing rules** so downstream rules fire correctly; admin dashboard Classification badge + Classify button (PR #34)
23. **NIfTI/BIDS conversion:** Python BIDS service (`bids-service/`) with dcm2niix backend; `require_bids_conversion` routing rule action; `bids_required` / `bids_status` study fields; DICOM→NIfTI conversion with BIDS-compliant directory structure + JSON sidecars; series-to-datatype classification; zip download endpoint; admin dashboard BIDS badge + Convert/Download buttons (PR #33)
24. **Batch import tools:** `api/cmd/import/` CLI + `POST /api/import/batch` API; recursive DICOM directory scan with `suyashkumar/dicom` header parsing; groups files by StudyInstanceUID; creates upload sessions + study records; evaluates routing rules; `--dry-run` mode (PR #31)
25. **MRI protocol compliance:** Python protocol service (`protocol-service/`) with pydicom-based parameter extraction for both Classic and Enhanced DICOM; `protocol_templates` table for per-project, per-manufacturer/model/software-version/sequence-type parameter rules with configurable tolerances and severity levels (critical/warning/info); `require_protocol_check` routing rule action; `protocol_required` / `protocol_status` study fields; async dispatch from Go API; admin dashboard Protocol badge + Check Protocol button + Protocol Templates CRUD tab (PR #41)
26. **Defacing tool upgrades:** DeepDefacer or afni_refacer for improved quality
27. **Authentication middleware:** Per-route auth middleware (`api/middleware/auth.go`) supporting GCP IAP and Azure AD Easy Auth; `RequireAuth` wrapper for admin routes; `RequireRole` for future viewer enforcement; `GET /api/auth/me` identity endpoint; `admin_users` lookup with case-insensitive email; dev mode auto-auth via `DEV_USER_EMAIL` ; all audit entries now record real user email; admin dashboard shows current user and handles 401/403 errors (PR #36)

28.  **Multi-cloud AWS support:** S3 storage backend (`api/storage/s3.go`) implementing the Storage interface with presigned URLs, copy-based move, and S3-compatible endpoint support (MinIO/LocalStack); AWS ALB + Cognito auth provider in middleware (JWT email extraction from `X-Amzn-0idc-Data` header); `S3_BUCKET / S3_REGION / S3_ENDPOINT` config; `terraform/aws/main.tf` with VPC, RDS PostgreSQL 15, S3, ECS Fargate cluster, ALB, ECR, KMS, SNS/SQS, CloudWatch (PR #50)

Future Ideas (not planned)

- DIMSE adapter for sites that can run an edge connector
- Non-DICOM formats: pathology whole-slide imaging, electron microscopy
- Client-side defacing preview via NiiVue (WebGL volume rendering)
- Modality-specific processing plugins (e.g., mammography CAD, cardiac segmentation)
- Federated learning integration

Key Open-Source Dependencies

Component	Library	License	Purpose
Go DICOM	suyashkumar/dicom	MIT	Backend DICOM validation
Browser DICOM read/write	dcmjs	MIT	Client-side tag modification
Browser DICOM parsing	dicomParser	MIT	Robust Part 10 parsing
Viewer	OHIF Viewer	MIT	DICOMweb viewer in admin
DICOM→NIfTI	dcm2niix	BSD	Format conversion for defacing
Defacing	mri_deface	Free	Facial feature removal
Python DICOM	pydicom	MIT	Pixel data injection in defacing svc
NIfTI I/O	nibabel	MIT	NIfTI reading in defacing svc
IaC	Terraform Google Provider	MPL-2.0	GCP infrastructure
Go PostgreSQL	pgx	MIT	Cloud SQL (PostgreSQL) driver
Go Migrations	goose	MIT	Database schema migrations
Vertex AI	cloud.google.com/go/aiplatform	Apache-2.0	Vertex AI SDK for Go

Verification Plan

After each phase, verify:

1. **Terraform:** `terraform plan` shows expected resources; `terraform apply` succeeds;
Healthcare API DICOM store accepts STOW-RS
2. **Go API:** Unit tests + integration test uploading a sample DICOM to GCS → Healthcare API
3. **Upload Portal:** Load a sample brain MRI DICOM directory, verify all PHI tags are stripped in the preview, upload succeeds
4. **Admin Dashboard:** View uploaded study in OHIF, verify de-identified tags
5. **Defacing:** Upload a head MRI → verify defacing service triggers → compare original vs defaced in viewer
6. **End-to-end:** External browser → upload → de-id → ingest → deface → review → approve → route